

MVP SaaS Templates – Sistema de Logs (ActivityLog)

Este documento descreve de forma técnica e operacional toda a implementação do sistema de logs do MVP SaaS Templates, incluindo arquitetura, modelos, controllers, queries, decisões de UX, problemas enfrentados e soluções aplicadas.

1. Objetivo do Sistema de Logs

Registrar eventos relevantes do sistema para auditoria, histórico operacional e UX, sem acoplamento direto às telas.

2. Migration – activity_logs

```
Schema::create('activity_logs', function (Blueprint $table) {
    $table->id();
    $table->string('event');
    $table->text('description')->nullable();
    $table->morphs('subject');
    $table->foreignId('user_id')->nullable()->constrained()->onDelete('nullOnDelete');
    $table->timestamps();
});
```

3. Model ActivityLog

```
class ActivityLog extends Model
{
    protected $fillable = [
        'event',
        'description',
        'subject_type',
        'subject_id',
        'user_id'
    ];

    public function subject()
    {
        return $this->morphTo();
    }

    public function user()
    {
        return $this->belongsTo(User::class);
    }
}
```

4. Método Centralizado ActivityLog::record

```
public static function record(
    string $event,
    string $description,
    Model $subject,
    ?User $user = null
) {
    self::create([
        'event' => $event,
```

```

        'description' => $description,
        'subject_type' => get_class($subject),
        'subject_id' => $subject->id,
        'user_id' => $user?->id,
    ]);
}

```

5. Uso no PublicationRequestController

```

$publicationRequest = PublicationRequest::create([
    'site_id' => $site->id,
    'status' => 'requested',
]);

ActivityLog::record(
    event: 'publication.requested',
    description: 'Solicitação de publicação criada pelo cliente',
    subject: $publicationRequest,
    user: auth()->user()
);

ActivityLog::record(
    event: 'site.publication.requested',
    description: 'Solicitação de publicação enviada',
    subject: $site,
    user: auth()->user()
);

```

6. Histórico no Editor do Site

```

public function activityLogs()
{
    return $this->morphMany(ActivityLog::class, 'subject')
        ->latest();
}

```

7. Dashboard – Query Filtrada (sem duplicação)

```

$activityLogs = ActivityLog::where('subject_type', Site::class)
    ->whereHasMorph('subject', [Site::class], function ($query) use ($user) {
        $query->where('client_id', $user->client_id);
    })
    ->latest()
    ->limit(10)
    ->get();

```

8. Comandos Utilizados

```

php artisan make:model ActivityLog -m
php artisan migrate

```

9. Problemas Enfrentados e Soluções

Foram enfrentados problemas como duplicação de logs na dashboard, uso incorreto de subject e variáveis não instanciadas. As soluções envolveram captura correta do retorno do Eloquent, duplicação consciente de logs e filtragem por subject_type na dashboard.

10. Estado Final

O sistema de logs está completo, consistente, extensível e alinhado com boas práticas de produtos SaaS, servindo tanto para UX quanto para auditoria.